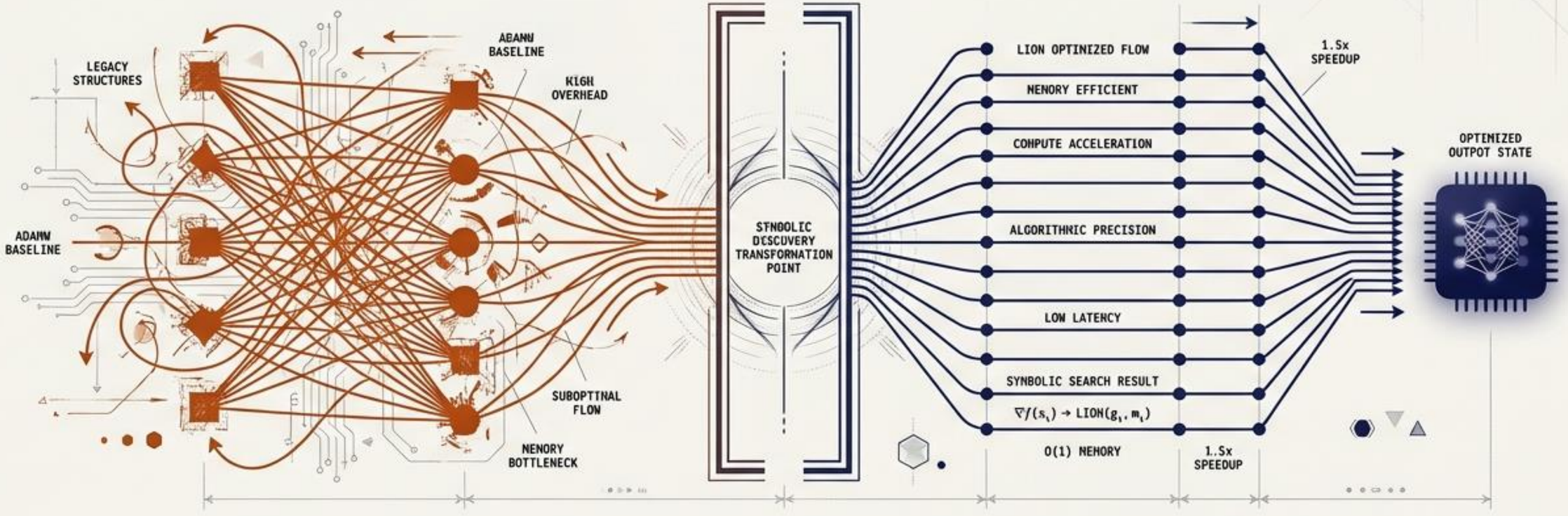
Lion: El Optimizador que **Desafía a AdamW**

Eficiencia de memoria y aceleración de cómputo descubiertas mediante búsqueda simbólica por IA.

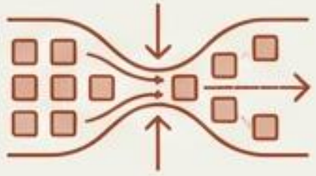


NotebookLM

El techo de cristal del entrenamiento moderno

El estándar indiscutible: AdamW

Requiere guardar tanto el primer momento (m_t) como el segundo momento (v_t) de los gradientes.



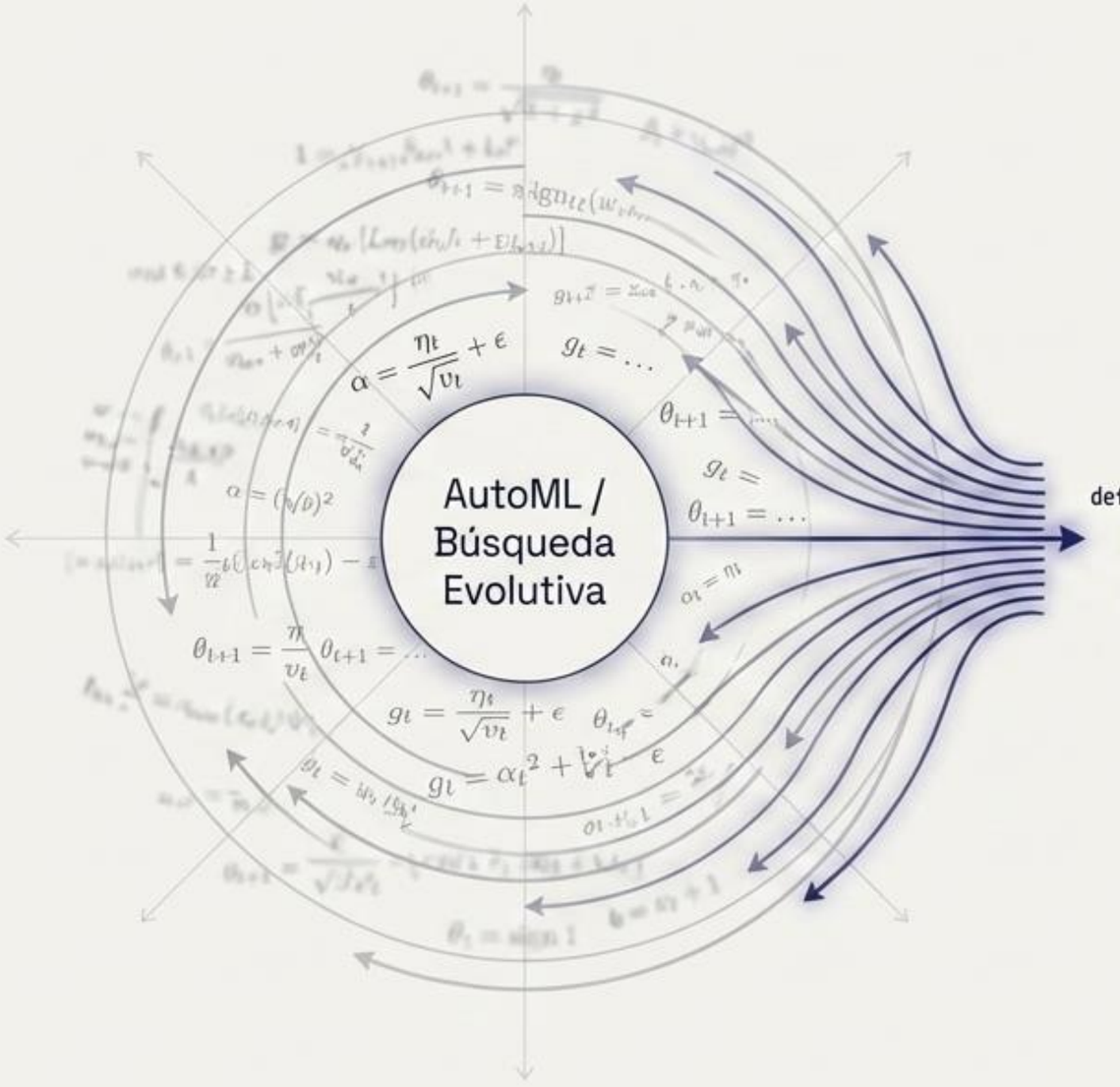
El Cuello de Botella

Duplica la huella de memoria adicional requerida por los parámetros.

A medida que los Transformers escalan a miles de millones de parámetros, la memoria del optimizador se convierte en el mayor obstáculo para incrementar el Tamaño de Lote (Batch Size).

NotebookLM

Inteligencia Artificial diseñando Inteligencia Artificial




```
def lion_step(params, grads, exp_avg, lr, beta1, beta2):
    exp_avg = beta1 * exp_avg + (1 - beta1) * grads
    p_update = sign(exp_avg)
    params = params - lr * p_update
    return params, exp_avg
```

Lion (EvoLved Sign Momentum) no fue diseñado por intuición humana.

Descubierto mediante la exploración algorítmica de un espacio infinito y disperso de programas matemáticos.

Representa un quiebre fundamental con las heurísticas tradicionales de optimización adaptativa.

NotebookLM

El Embudo Evolutivo

Etapa 1: Espacio Infinito

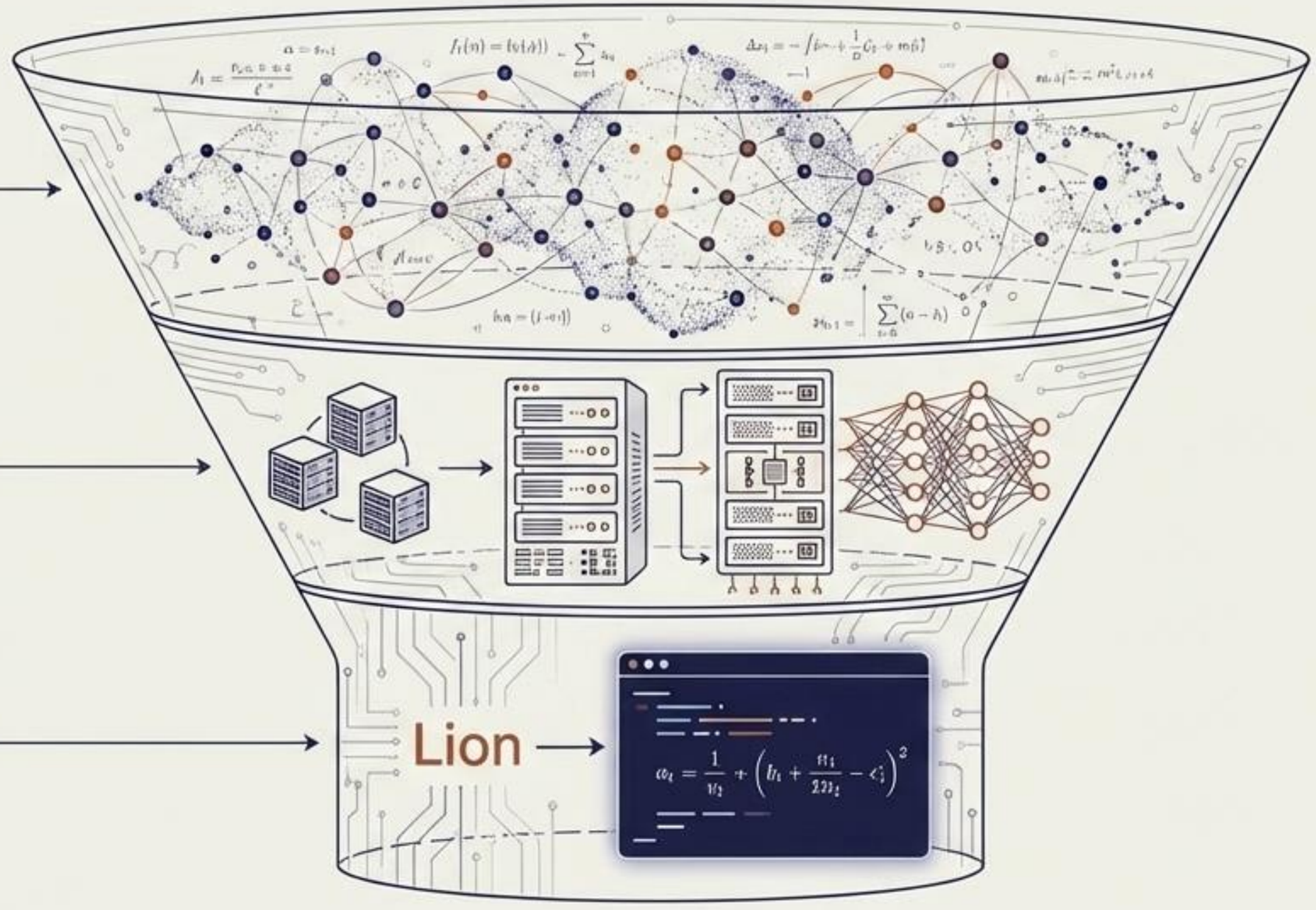
Generación de programas con mutaciones regulares, variables matemáticas y ejecución abstracta (Filtrado de errores y hashing).

Etapa 2: Meta-Validación

Selección en tareas proxy pequeñas, validando la generalización hacia arquitecturas masivas (ViT, ResNet, Transformers).

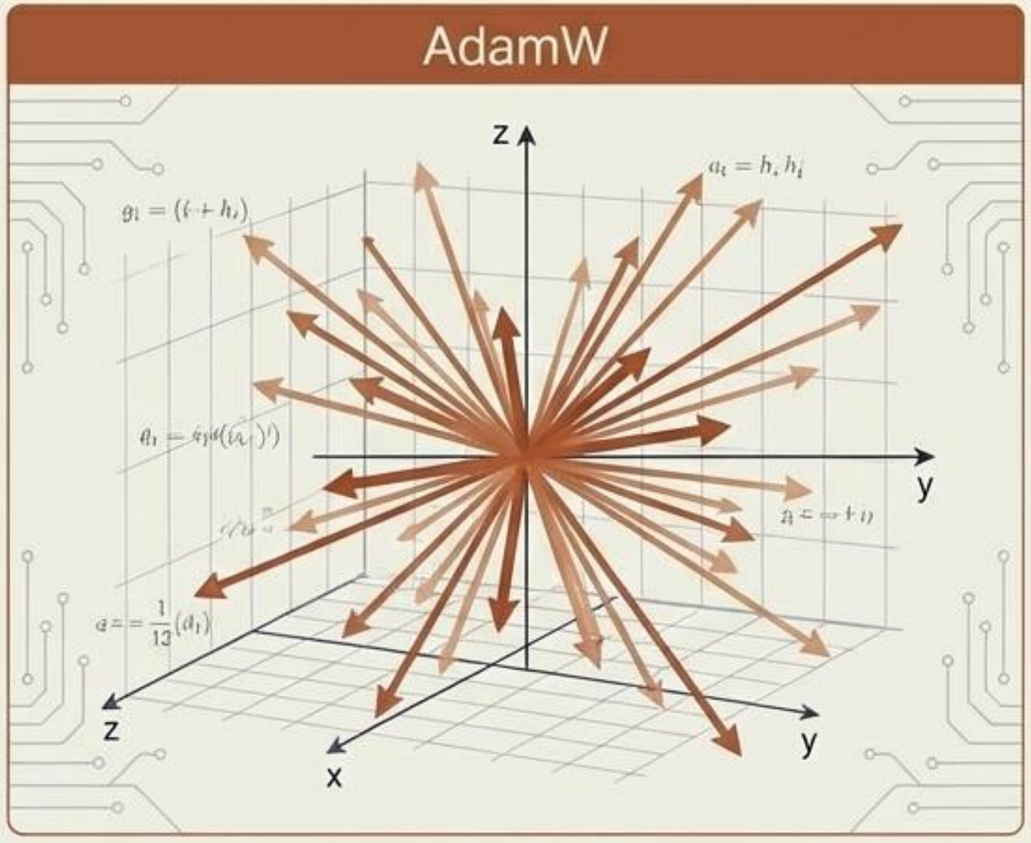
Etapa 3: Simplificación

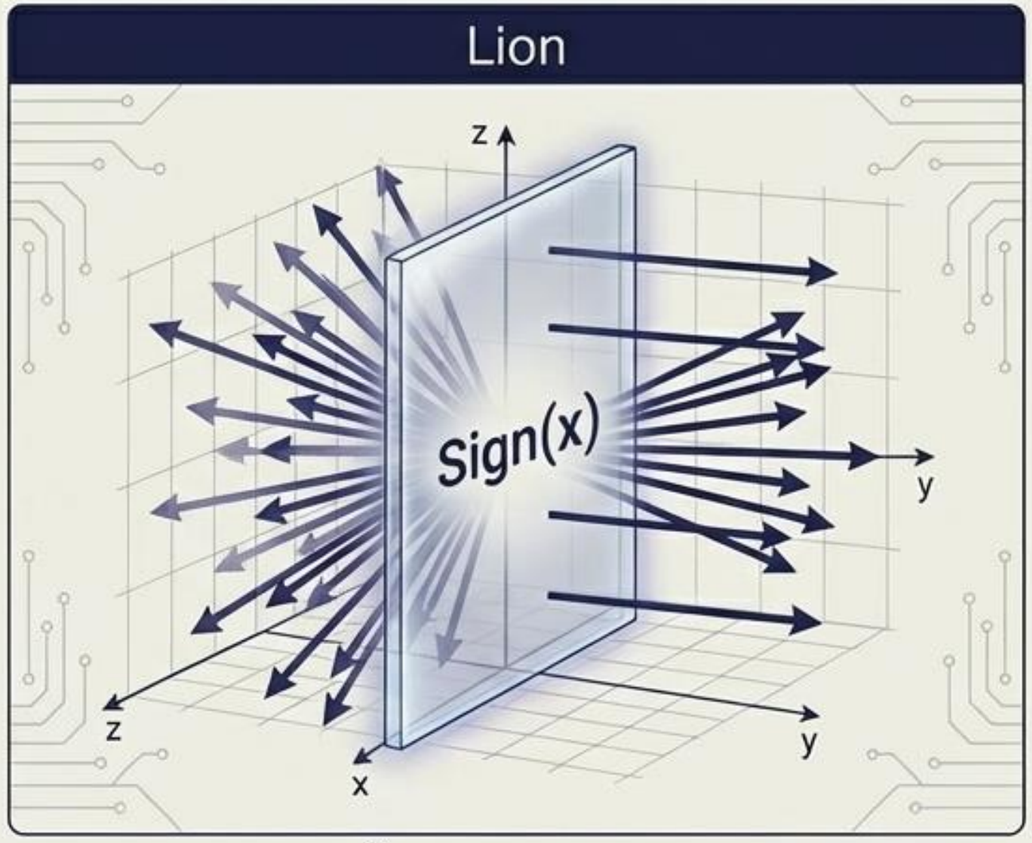
Eliminación de redundancias. Extracción de la fórmula matemática óptima: Lion.



NotebookLM

La Operación de Signo: Filtro de Magnitud Uniforme



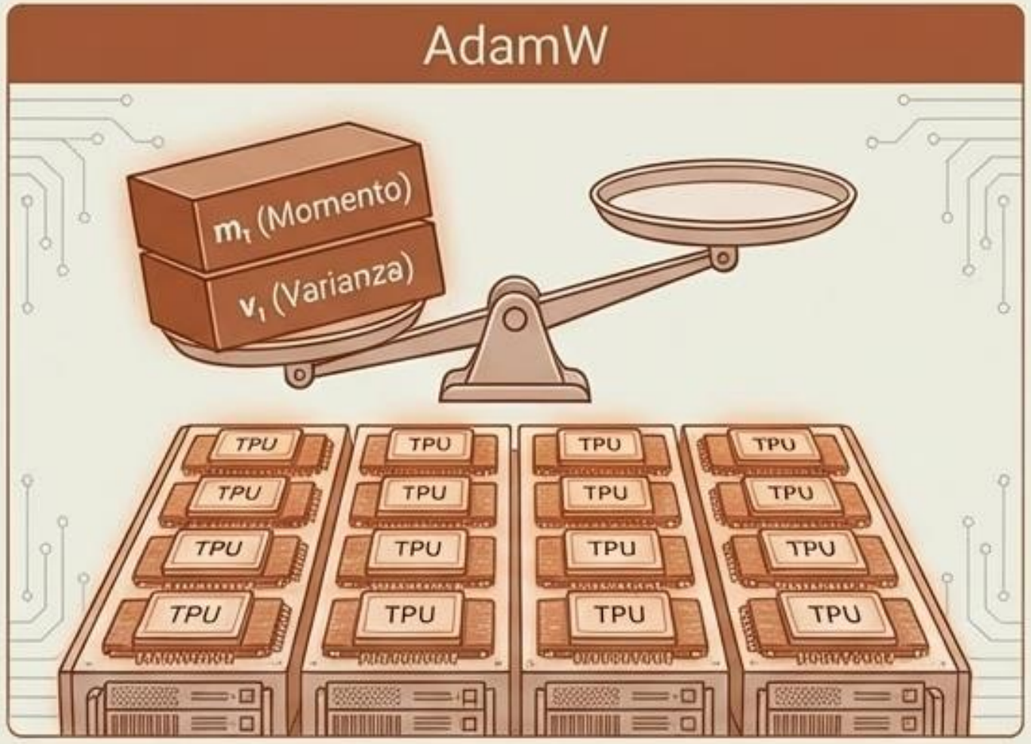


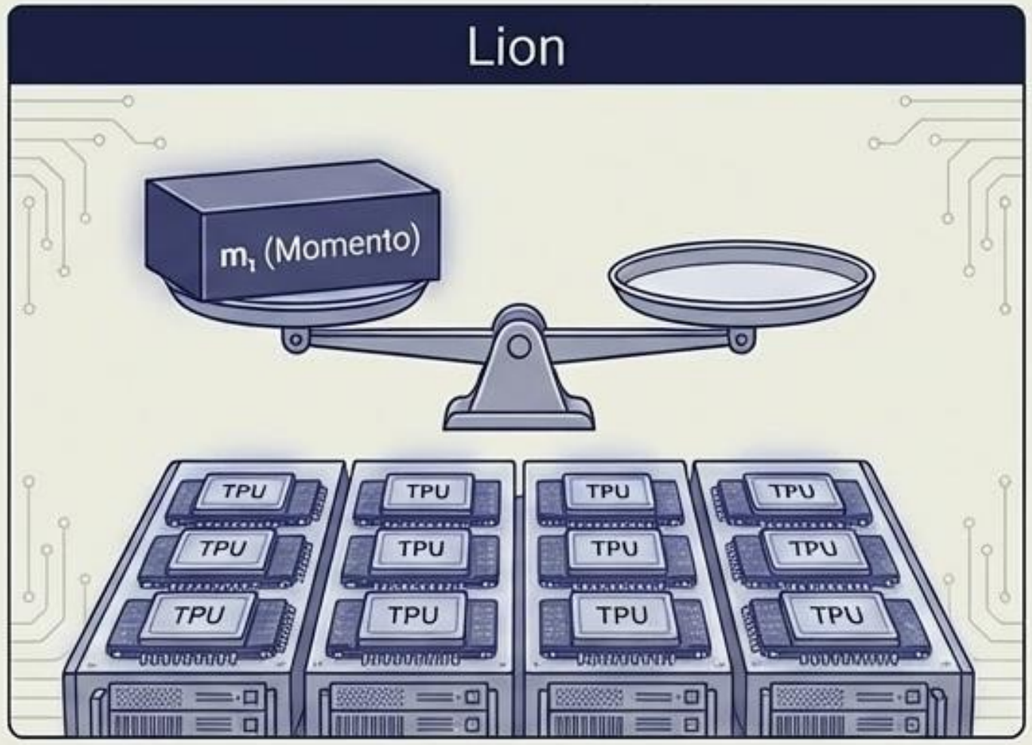
Conclusión: La operación de signo actúa como un regularizador integrado (añadiendo ruido beneficioso) y garantiza actualizaciones de magnitud constante en cada paso, independientemente de la varianza del gradiente.

NotebookLM

Reducción drástica de la huella de memoria

Objetivo: Entrenar ViT-B/16 con Batch Size de 4096





Conclusión: Al eliminar el seguimiento del segundo momento, Lion reduce a la mitad el estado de memoria del optimizador, liberando espacio crítico para escalar el procesamiento de datos paralelos.

NotebookLM

Matriz de Diagnóstico: El Enfrentamiento de Optimizadores

Métrica	AdamW (El Estándar)	Lion (El Retador)
Origen	Intuición Humana	Búsqueda Simbólica por IA
Rastreo de Memoria	Momento + Varianza (Alta)	Solo Momento (Baja)
Cálculo de Actualización	Magnitud Adaptativa	Operación de Signo Uniforme
Costo de Hardware Ejemplo	16 chips TPU v4	8 chips TPU v4
Velocidad de Ejecución	Línea base (1x)	+2-15% más rápido (Pasos / seg)
Preferencia de Batch	Estándar	Masivo / Data Parallelism

NotebookLM

Comparación de pseudoalgoritmos

Algorithm 1 AdamW Optimizer

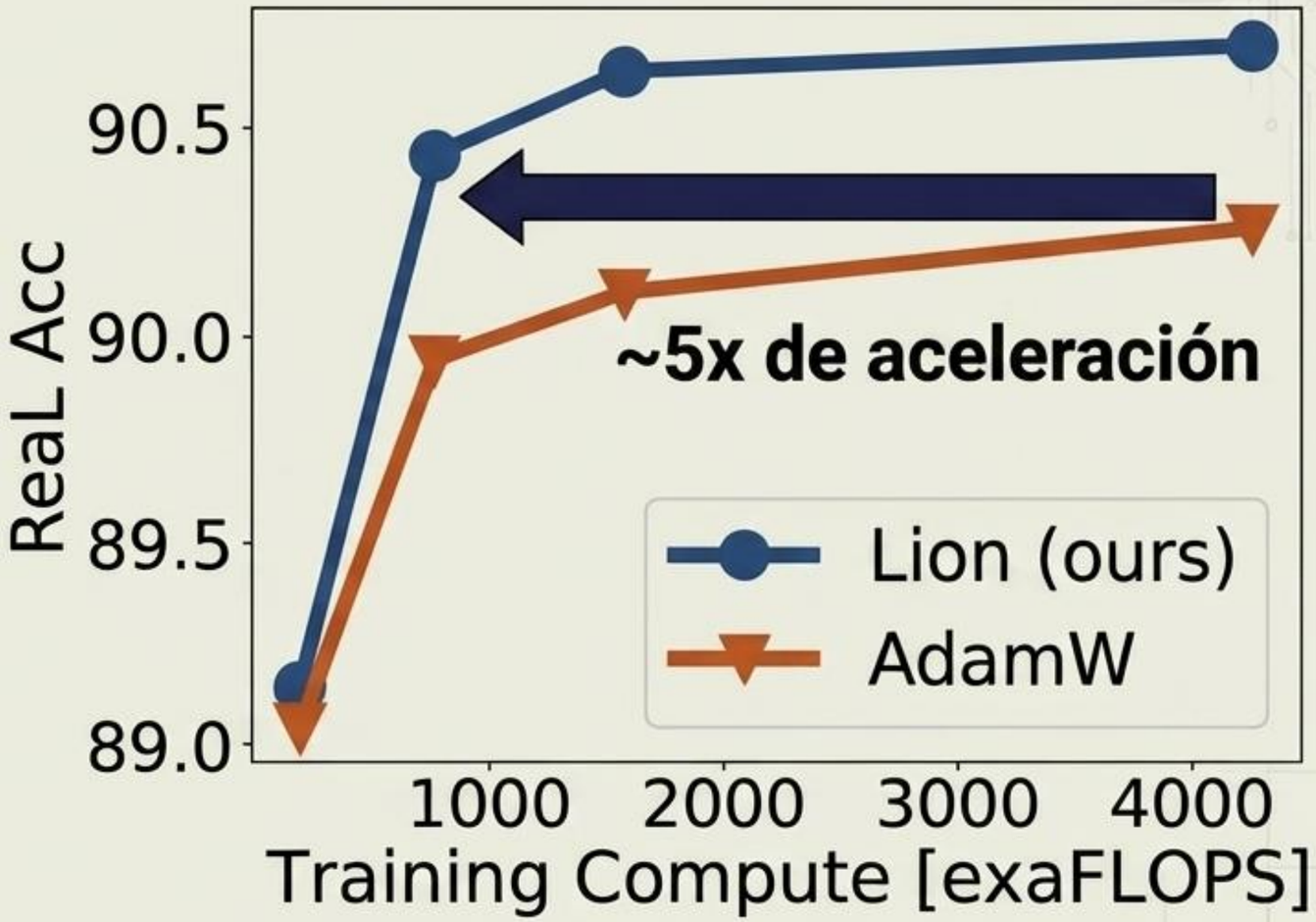
given $\beta_1, \beta_2, \epsilon, \lambda, \eta, f$
initialize θ_0, $m_0 \leftarrow 0$, $v_0 \leftarrow 0$, $t \leftarrow 0$
while θ_t not converged **do**
  $t \leftarrow t + 1$
  $g_t \leftarrow \nabla_\theta f(\theta_{t-1})$
  update EMA of g_t **and** g_t^2
  $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$
  $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
  bias correction
  $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$
  $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$
  update model parameters
  $\theta_t \leftarrow \theta_{t-1} - \eta_t (\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda \theta_{t-1})$
end while
return θ_t

Algorithm 2 Lion Optimizer (ours)

given $\beta_1, \beta_2, \lambda, \eta, f$
initialize θ_0, $m_0 \leftarrow 0$
while θ_t not converged **do**
  $g_t \leftarrow \nabla_\theta f(\theta_{t-1})$
  update model parameters
  $c_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$
  $\theta_t \leftarrow \theta_{t-1} - \eta_t (\text{sign}(c_t) + \lambda \theta_{t-1})$
  update EMA of g_t
  $m_t \leftarrow \beta_2 m_{t-1} + (1 - \beta_2) g_t$
end while
return θ_t

Visión por Computadora: 5x de Ahorro en Cómputo

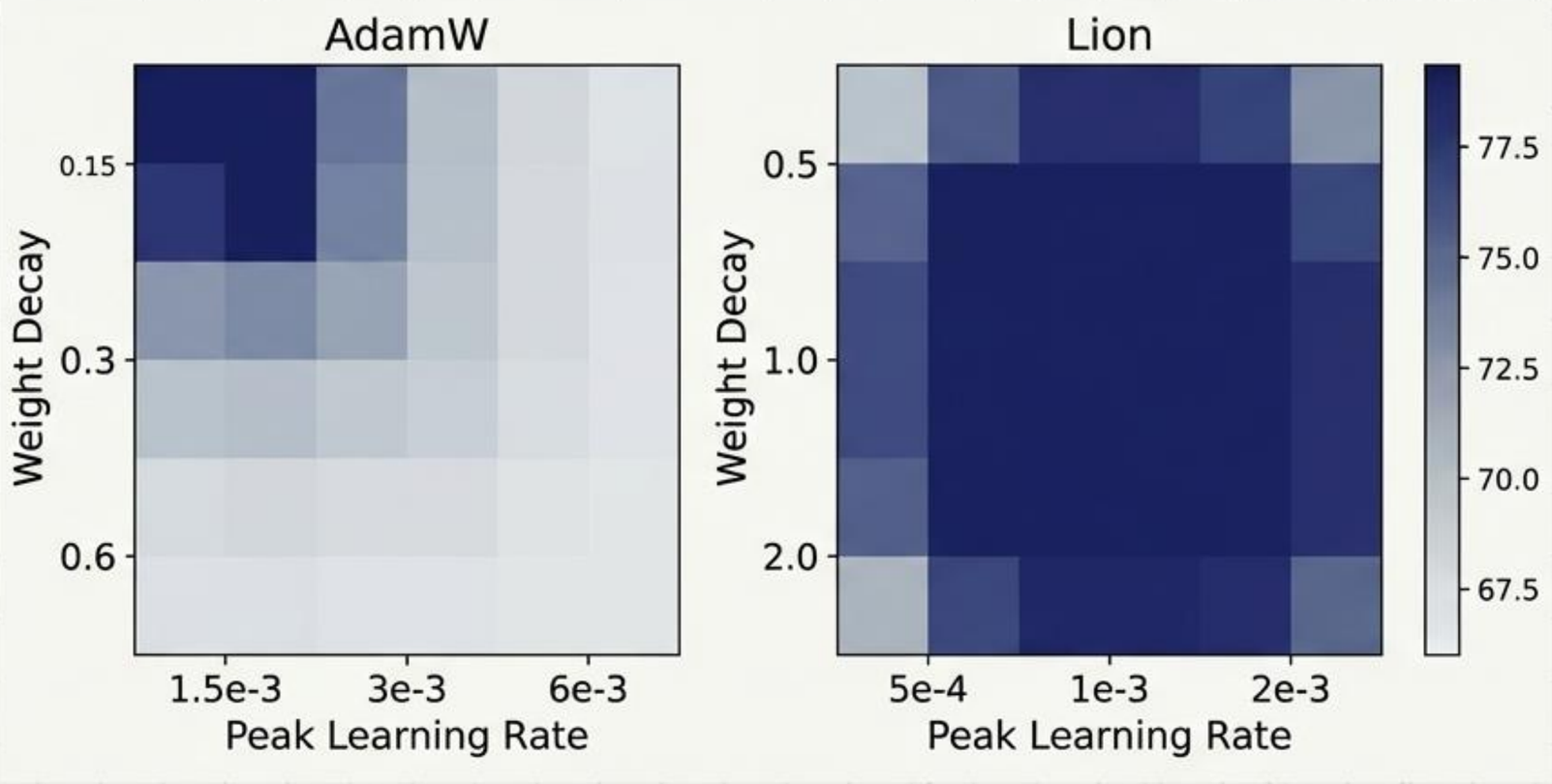
Entrenar arquitecturas ViT masivas en JFT-300M requiere recursos astronómicos. Lion alcanza el 90.3% de precisión utilizando solo una **quinta parte** del presupuesto de cómputo necesario para AdamW.



NotebookLM

Tolerancia a Hiperparámetros

Contrario a la intuición de que un optimizador sin segundo momento sería inestable, Lion demuestra ser significativamente más tolerante a configuraciones imperfectas, requiriendo menos barridos computacionalmente costosos.



NotebookLM

```python
# toy model

import torch
from torch import nn

model = nn.Linear(10, 1)

# import Lion and instantiate with parameters

from lion_pytorch import Lion

opt = Lion(model.parameters(), lr=1e-4, weight_decay=1e-2)

# forward and backwards

loss = model(torch.randn(10))
loss.backward()

# optimizer step

opt.step()
opt.zero_grad()
```